MySQL User-Defined Functions (UDF)

1. Introduction to Functions

A **function** is a reusable block of SQL logic that:

1. Accepts zero or more input values.
2. Performs some operation.
3. Returns **one value**.

For example:

```
SELECT UPPER('mysql');
```

Output:

```
MYSQL
```

`UPPER()` is a **built-in MySQL function**.

Sometimes the built-in functions are not sufficient for a particular business requirement. In that case, we can create our own function.

Such a function is called a **User-Defined Function (UDF)**.

---

2. What is a UDF?

UDF = User-Defined Function

A UDF is a function created by the developer/database administrator to perform a specific operation according to a business requirement.

Simple example

Suppose we frequently need to calculate annual salary:

```
Annual Salary = Monthly Salary × 12
```

Instead of writing this calculation everywhere:

```
salary * 12
```

we can create a function:

```
CREATE FUNCTION annual_salary(...)
```

and then use:

```
SELECT annual_salary(50000);
```

Output:

```
600000
```

Definition

A User-Defined Function in MySQL is a stored program that accepts input parameters, performs a specific operation, and returns a single value.

---

3. Why Do We Need UDFs?

Suppose you have 20 different queries where you need to calculate annual salary.

Without a function:

```
SELECT salary * 12 FROM employees;
```

You may repeat the same business logic in many places.

With a function:

```
SELECT annual_salary(salary)
FROM employees;
```

Now the calculation is written once and reused.

Main advantages

1. Reusability

Write the logic once and use it multiple times.

2. Readability

This:

```
annual_salary(salary)
```

is easier to understand than:

```
salary * 12
```

3. Maintainability

If the calculation changes, you can modify the function instead of modifying many queries.

4. Consistency

The same calculation is performed in the same way everywhere.

5. Reduced code duplication

Repeated logic can be centralized.

---

4. Built-in Function vs User-Defined Function

MySQL already provides many functions.

Examples:

```
UPPER()
LOWER()
LENGTH()
ROUND()
SUM()
```

```
AVG()
COUNT()
NOW()
CONCAT()
```

These are **built-in functions**.

We can create functions such as:

```
annual_salary()
calculate_bonus()
get_grade()
calculate_discount()
employee_category()
```

These are **user-defined functions**.

Built-in Function	User-Defined Function
Provided by MySQL	Created by developer
Already available	Must be created
General-purpose	Business-specific
Example: `UPPER()`	Example: `get_grade()`
Example: `ROUND()`	Example: `calculate_bonus()`

---

5. Basic Structure of a UDF

The basic syntax is:

```
CREATE FUNCTION function_name(parameter_name datatype)
RETURNS return_datatype
BEGIN

    -- function logic

    RETURN value;

END;
```

For a multi-statement function, we normally change the delimiter:

```
DELIMITER //

CREATE FUNCTION function_name(parameter_name datatype)
RETURNS return_datatype
BEGIN

    -- statements

    RETURN value;

END //

DELIMITER ;
```

---

6. Understanding Every Part of the Syntax

Consider:

```
DELIMITER //

CREATE FUNCTION annual_salary(monthly_salary DECIMAL(10,2))
```

```
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    RETURN monthly_salary * 12;
END //

DELIMITER ;
```

Let's understand each part.

6.1 `DELIMITER`

Normally MySQL uses:

`;`

as the statement terminator.

Inside a function, we may have multiple statements ending with `;`.

For example:

```
BEGIN
    DECLARE x INT;
    SET x = 10;
    RETURN x;
END;
```

If the normal delimiter is active, MySQL may interpret the first semicolon as the end of the `CREATE FUNCTION` statement.

Therefore we temporarily change the delimiter:

```
DELIMITER //
```

Now the function ends with:

```
END //
```

After creating it:

```
DELIMITER ;
```

returns the delimiter to normal.

Important point

`DELIMITER` is mainly a **MySQL client command** used to help send the complete routine definition. It is not part of the function's stored logic.

---

7. `CREATE FUNCTION`

The keyword:

```
CREATE FUNCTION
```

tells MySQL that we want to create a stored function.

Example:

```
CREATE FUNCTION annual_salary(...)
```

---

8. Function Name

The name identifies the function.

Example:

```
annual_salary
```

Other examples:

```
calculate_bonus
get_grade
calculate_discount
get_full_name
```

A function name should clearly describe what the function does.

Good naming

```
calculate_bonus()
annual_salary()
get_grade()
```

Poor naming

```
abc()
fun1()
test()
```

Use meaningful names.

---

9. Parameters

Parameters are values supplied to the function.

Example:

```
monthly_salary DECIMAL(10,2)
```

Here:

```
monthly_salary = parameter name
DECIMAL(10,2) = parameter datatype
```

A function can have:

- Zero parameters
- One parameter
- Multiple parameters

9.1 Function With One Parameter

```
CREATE FUNCTION square_num(num INT)
RETURNS INT
```

```
...
```

Here:

`num`

is the parameter.

9.2 Function With Multiple Parameters

```
CREATE FUNCTION add_numbers(a INT, b INT)
RETURNS INT
...
```

Here there are two parameters:

`a`
`b`

---

10. `RETURNS`

`RETURNS` specifies the datatype of the value that the function will return.

Example:

```
RETURNS INT
```

means the function returns an integer.

Another example:

```
RETURNS DECIMAL(10,2)
```

means the function returns a decimal value.

Another:

```
RETURNS VARCHAR(50)
```

means it returns text.

Examples

```
RETURNS INT
RETURNS DECIMAL(10,2)
RETURNS VARCHAR(100)
RETURNS DATE
```

---

11. `RETURN`

`RETURN` sends the final result back to the caller.

Example:

```
RETURN num * num;
```

If:

```
num = 5
```

the function returns:

```
25
```

Very important

A MySQL stored function must return a value compatible with its declared `RETURNS` datatype.

For example:

```
RETURNS INT
```

should return an integer-compatible result.

---

12. First UDF Example

Let's create a simple function to calculate square.

```
DELIMITER //

CREATE FUNCTION square_num(num INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN num * num;
END //

DELIMITER ;
```

Call the function:

```
SELECT square_num(5);
```

Output:

```
25
```

Another example:

```
SELECT square_num(10);
```

Output:

```
100
```

---

13. Function Without `BEGIN...END`

When the function contains only one expression, we can write it more simply:

```
DELIMITER //

CREATE FUNCTION square_num(num INT)
RETURNS INT
DETERMINISTIC
RETURN num * num //

DELIMITER ;
```

This is shorter.

When to use `BEGIN...END`

Use `BEGIN...END` when you need multiple statements, variables, conditions, etc.

For example:

```
BEGIN
    DECLARE bonus DECIMAL(10,2);

    SET bonus = salary * 0.10;

    RETURN bonus;
END
```

---

14. Calling a User-Defined Function

After creating the function, we can call it using `SELECT`.

Example:

```
SELECT square_num(5);
```

We can also call it with an expression:

```
SELECT square_num(5 + 2);
```

MySQL first evaluates:

```
5 + 2 = 7
```

and then:

```
square_num(7) = 49
```

---

15. UDF With Multiple Parameters

Let's create a function to calculate total price.

Formula:

```
Total = Price × Quantity
```

```
DELIMITER //

CREATE FUNCTION calculate_total(
    price DECIMAL(10,2),
    quantity INT
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    RETURN price * quantity;
END //

DELIMITER ;
```

Call:

```
SELECT calculate_total(500, 3);
```

Output:

```
1500.00
```

---

16. UDF Returning a String

A function doesn't have to return a number.

Example:

```
DELIMITER //

CREATE FUNCTION get_full_name(
    first_name VARCHAR(50),
    last_name VARCHAR(50)
)
RETURNS VARCHAR(100)
DETERMINISTIC
BEGIN
    RETURN CONCAT(first_name, ' ', last_name);
END //

DELIMITER ;
```

Call:

```
SELECT get_full_name('Akshat', 'Saini');
```

Output:

```
Akshat Saini
```

---

17. UDF With `IF...ELSE`

We can use conditions inside a function.

Suppose:

```
Salary >= 50000 → High
Salary < 50000  → Normal
```

Function:

```
DELIMITER //

CREATE FUNCTION salary_category(
    salary_amount DECIMAL(10,2)
)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN

    IF salary_amount >= 50000 THEN
        RETURN 'High';
    ELSE
        RETURN 'Normal';
    END IF;

END //

DELIMITER ;
```

Call:

```
SELECT salary_category(60000);
```

Output:

```
High
```

Call:

```
SELECT salary_category(40000);
```

Output:

```
Normal
```

---

18. UDF With `IF...ELSEIF...ELSE`

Multiple conditions can be used.

```
DELIMITER //

CREATE FUNCTION salary_grade(
    salary_amount DECIMAL(10,2)
)
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN

    IF salary_amount >= 70000 THEN
        RETURN 'A';

    ELSEIF salary_amount >= 50000 THEN
        RETURN 'B';

    ELSEIF salary_amount >= 30000 THEN
        RETURN 'C';

    ELSE
        RETURN 'D';

    END IF;

END //

DELIMITER ;
```

Test:

```
SELECT salary_grade(80000);
```

Output:

```
A
```

```
SELECT salary_grade(55000);
```

Output:

```
B
```

```
SELECT salary_grade(35000);
```

Output:

```
C
```

---

19. UDF With `CASE`

Instead of `IF`, we can use `CASE`.

```
DELIMITER //

CREATE FUNCTION get_grade(mark INT)
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN

    RETURN CASE
        WHEN mark >= 90 THEN 'A+'
        WHEN mark >= 80 THEN 'A'
        WHEN mark >= 70 THEN 'B'
        WHEN mark >= 60 THEN 'C'
        WHEN mark >= 50 THEN 'D'
        ELSE 'F'
    END;

END //

DELIMITER ;
```

Call:

```
SELECT get_grade(85);
```

Output:

```
A
```

---

20. UDF With Local Variables

Sometimes we need to store intermediate results.

For this, we use:

```
DECLARE
```

Example:

```
DELIMITER //

CREATE FUNCTION calculate_bonus(
    salary_amount DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN

    DECLARE bonus DECIMAL(10,2);

    SET bonus = salary_amount * 0.10;

    RETURN bonus;

END //

DELIMITER ;
```

Call:

```
SELECT calculate_bonus(50000);
```

Output:

```
5000.00
```

Flow

```
salary_amount
      ↓
salary × 10%
      ↓
bonus variable
      ↓
RETURN bonus
```

---

21. DECLARE

`DECLARE` creates a local variable inside the function.

Syntax:

```
DECLARE variable_name datatype;
```

Example:

```
DECLARE bonus DECIMAL(10,2);
```

Another:

```
DECLARE total DECIMAL(10,2);
```

Another:

```
DECLARE result INT;
```

Important rule

Declarations such as `DECLARE` must appear at the beginning of the block, before executable statements such as `SET`, `SELECT`, or `RETURN`.

---

22. SET

`SET` assigns a value to a variable.

Example:

```
DECLARE bonus DECIMAL(10,2);

SET bonus = salary * 0.10;
```

Here:

```
DECLARE → creates variable
SET     → assigns value
RETURN  → returns value
```

---

23. DECLARE, SET, and RETURN

This sequence is extremely important for students.

```
DECLARE total DECIMAL(10,2);

SET total = price * quantity;
```

```
RETURN total;
```

Think:

```
DECLARE = Create a box
SET     = Put value into the box
RETURN  = Send the value outside
```

---

24. Using UDF With a Table

Let's create an employee table.

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100),
    salary DECIMAL(10,2)
);
```

Insert data:

```
INSERT INTO employees
VALUES
(1, 'Rahul', 40000),
(2, 'Priya', 50000),
(3, 'Amit', 60000),
(4, 'Neha', 45000);
```

Create annual salary function:

```
DELIMITER //

CREATE FUNCTION annual_salary(
    monthly_salary DECIMAL(10,2)
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    RETURN monthly_salary * 12;
END //

DELIMITER ;
```

Now use it with the table:

```
SELECT
    employee_name,
    salary,
    annual_salary(salary) AS annual_salary
FROM employees;
```

Result:

employee_name	salary	annual_salary
Rahul	40000	480000
Priya	50000	600000
Amit	60000	720000
Neha	45000	540000

This demonstrates the real power of a UDF.

---

25. UDF in `WHERE`

A function can also be used inside conditions.

Example:

```
SELECT
    employee_name,
    salary
FROM employees
WHERE annual_salary(salary) > 600000;
```

This means:

```
Calculate annual salary
        ↓
Check whether > 600000
        ↓
Return matching employees
```

---

26. UDF in `ORDER BY`

Example:

```
SELECT
    employee_name,
    salary,
    annual_salary(salary) AS yearly_salary
FROM employees
ORDER BY annual_salary(salary) DESC;
```

Employees will be sorted according to annual salary.

---

27. UDF and `NULL`

Students should understand that `NULL` can affect function results.

Example:

```
SELECT annual_salary(NULL);
```

The result may be:

```
NULL
```

because:

```
NULL * 12
```

is:

```
NULL
```

If your business logic requires special handling of `NULL`, you can explicitly handle it.

Example:

```
DELIMITER //

CREATE FUNCTION safe_annual_salary(
```

```
    monthly_salary DECIMAL(10,2)
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN

    IF monthly_salary IS NULL THEN
        RETURN 0;
    END IF;

    RETURN monthly_salary * 12;

END //

DELIMITER ;
```

Now:

```
SELECT safe_annual_salary(NULL);
```

returns:

```
0
```

---

28. What is `DETERMINISTIC`?

`DETERMINISTIC` is a **routine characteristic**.

It tells MySQL that:

> Given the same input values, the function will always produce the same result.

Example:

```
DELIMITER //

CREATE FUNCTION square_num(num INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN num * num;
END //

DELIMITER ;
```

For:

```
SELECT square_num(5);
```

the result is always:

```
25
```

So the function is deterministic.

---

29. Deterministic Example

Consider:

```
salary * 12
```

For:

```
salary = 50000
```

the answer is always:

```
600000
```

Therefore this is a deterministic calculation.

```
CREATE FUNCTION annual_salary(salary DECIMAL(10,2))
RETURNS DECIMAL(12,2)
DETERMINISTIC
RETURN salary * 12;
```

---

30. Non-Deterministic Concept

A function can be non-deterministic when its result can change even though the explicit inputs are the same.

Examples of changing information include:

```
NOW()
CURRENT_TIMESTAMP
RAND()
```

For example:

```
SELECT NOW();
```

The returned time changes.

Therefore a function based on the current time is not a predictable fixed result for the same input/context.

---

31. What Happens If We Don't Write `DETERMINISTIC`?

This is an important student question.

Consider:

```
CREATE FUNCTION square_num(num INT)
RETURNS INT
BEGIN
    RETURN num * num;
END;
```

The function may still work normally.

You can call:

```
SELECT square_num(5);
```

and get:

```
25
```

Then why write `DETERMINISTIC`?

Because it explicitly declares the function's behavior to MySQL.

Compare:

```
DETERMINISTIC
```

with:

```
NOT DETERMINISTIC
```

These describe the routine's characteristics.

Important teaching point

`DETERMINISTIC` does **not** perform the calculation.

This:

```
RETURN num * num;
```

performs the calculation.

This:

```
DETERMINISTIC
```

describes the expected behavior of the function.

---

32. `DETERMINISTIC` in Simple Language

Teach students this shortcut:

```
Same input → Same output
        ↓
DETERMINISTIC
```

Example:

```
5 → 25
5 → 25
5 → 25
```

Predictable.

But if the result depends on changing information:

```
same context + current time → different result at different times
```

it may be non-deterministic.

---

33. Can a Function Have No Parameters?

Yes.

For example, a function can return a fixed value or derive a value from allowed context.

Example:

```
DELIMITER //

CREATE FUNCTION company_name()
RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
    RETURN 'ABC Technologies';
END //
```

```
DELIMITER ;
```

Call:

```
SELECT company_name();
```

Output:

```
ABC Technologies
```

However, whether a no-parameter function is appropriate depends on the business requirement.

---

34. Can a Function Have Multiple Parameters?

Yes.

Example:

```
DELIMITER //

CREATE FUNCTION calculate_discount(
    amount DECIMAL(10,2),
    discount_percent DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN

    RETURN amount * discount_percent / 100;

END //

DELIMITER ;
```

Call:

```
SELECT calculate_discount(10000, 10);
```

Output:

```
1000.00
```

---

35. Practical Example: Discount Calculation

Suppose a company has this rule:

```
Amount >= 10000 → 20% discount
Amount >= 5000  → 10% discount
Otherwise       → 5% discount
```

Function:

```
DELIMITER //

CREATE FUNCTION calculate_discount(
    amount DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
```

```
    IF amount >= 10000 THEN
        RETURN amount * 0.20;

    ELSEIF amount >= 5000 THEN
        RETURN amount * 0.10;

    ELSE
        RETURN amount * 0.05;

    END IF;

END //

DELIMITER ;
```

Test:

```
SELECT calculate_discount(12000);
```

Output:

```
2400.00
```

---

36. Practical Example: Final Price

Now let's calculate the price after discount.

```
DELIMITER //

CREATE FUNCTION final_price(
    amount DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN

    DECLARE discount DECIMAL(10,2);

    IF amount >= 10000 THEN
        SET discount = amount * 0.20;

    ELSEIF amount >= 5000 THEN
        SET discount = amount * 0.10;

    ELSE
        SET discount = amount * 0.05;

    END IF;

    RETURN amount - discount;

END //

DELIMITER ;
```

Call:

```
SELECT final_price(12000);
```

Calculation:

```
12000
- 2400
------
9600
```

Result:

```
9600.00
```

---

37. Practical Example: Student Grade

Create table:

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100),
    marks INT
);
```

Insert data:

```
INSERT INTO students
VALUES
(1, 'Rahul', 85),
(2, 'Priya', 72),
(3, 'Amit', 91),
(4, 'Neha', 65),
(5, 'Rohan', 45);
```

Create function:

```
DELIMITER //

CREATE FUNCTION get_grade(mark INT)
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN

    IF mark >= 90 THEN
        RETURN 'A+';

    ELSEIF mark >= 80 THEN
        RETURN 'A';

    ELSEIF mark >= 70 THEN
        RETURN 'B';

    ELSEIF mark >= 60 THEN
        RETURN 'C';

    ELSEIF mark >= 50 THEN
        RETURN 'D';

    ELSE
        RETURN 'F';

    END IF;

END //

DELIMITER ;
```

Use:

```
SELECT
    student_id,
    student_name,
    marks,
    get_grade(marks) AS grade
FROM students;
```

Result:

Student	Marks	Grade
Rahul	85	A
Priya	72	B
Amit	91	A+
Neha	65	C
Rohan	45	F

---

38. UDF vs Stored Procedure

This is one of the most important topics.

Function	Procedure
Created using `CREATE FUNCTION`	Created using `CREATE PROCEDURE`
Must return a value	Does not have to return a value
Uses `RETURN`	Uses result sets and/or OUT parameters for outputs
Has `RETURNS` clause	No `RETURNS` clause
Can be used in expressions	Called separately
Commonly used for calculations	Commonly used for operations/workflows
Can be used in `SELECT` expressions	Called using `CALL`

Function

```
SELECT annual_salary(50000);
```

Procedure

```
CALL show_employees();
```

---

39. Function Example

```
DELIMITER //

CREATE FUNCTION add_numbers(a INT, b INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN a + b;
END //

DELIMITER ;
```

Call:

```
SELECT add_numbers(10, 20);
```

Result:

```
30
```

---

40. Procedure Example

```
DELIMITER //

CREATE PROCEDURE show_employees()
BEGIN
    SELECT * FROM employees;
END //

DELIMITER ;
```

Call:

```
CALL show_employees();
```

The procedure executes the query.

Easy way to remember

```
FUNCTION
    ↓
Give me a value

PROCEDURE
    ↓
Perform an operation
```

---

41. Important Restrictions and Design Considerations

A stored function has restrictions that do not apply in exactly the same way to ordinary application code.

For example, stored functions cannot perform transaction-control statements such as:

```
COMMIT
ROLLBACK
```

and MySQL imposes restrictions on modifying data and other statements inside stored functions.

Therefore, functions are generally best for:

```
Calculations
Classification
Formatting
Transformations
Reusable expressions
```

while procedures are often better for:

```
Multi-step operations
Business workflows
Data-processing tasks
Operations that involve several SQL statements
```

---

42. Viewing Functions

To view stored functions:

```
SHOW FUNCTION STATUS;
```

To view functions for a specific database:

```
SHOW FUNCTION STATUS
```

```
WHERE Db = 'company_db';
```

---

43. View the Definition of a Function

Use:

```
SHOW CREATE FUNCTION annual_salary;
```

This displays the function definition stored in MySQL.

This is useful when:

- Checking existing logic
- Debugging
- Understanding an old database
- Reviewing code

---

44. Deleting a Function

To remove a function:

```
DROP FUNCTION annual_salary;
```

Safer:

```
DROP FUNCTION IF EXISTS annual_salary;
```

Important

`DROP FUNCTION` permanently removes that stored function definition from the database.

---

45. Modifying a Function

If the function logic needs to change, commonly you remove/recreate the function definition.

For example:

```
DROP FUNCTION IF EXISTS annual_salary;
```

Then create it again with the new logic.

Always be careful when changing functions used by production queries.

---

46. Common Errors Students Make

Error 1: Using `RETURN` incorrectly

Incorrect concept:

```
RETURN salary * 12;
```

while declaring:

```
RETURNS VARCHAR(20)
```

The declared return type and returned value should be compatible.

Error 2: Forgetting `RETURNS`

Incorrect:

```
CREATE FUNCTION annual_salary(salary DECIMAL(10,2))
RETURN DECIMAL(12,2)
```

Correct:

```
CREATE FUNCTION annual_salary(salary DECIMAL(10,2))
RETURNS DECIMAL(12,2)
```

Error 3: Forgetting `RETURN`

A function must actually return a value.

Error 4: Incorrect delimiter handling

For multi-statement routines:

```
DELIMITER //

CREATE FUNCTION ...
BEGIN
    ...
END //

DELIMITER ;
```

Error 5: Declaring variables after executable statements

Incorrect:

```
BEGIN
    SET x = 10;
    DECLARE x INT;
END
```

Correct:

```
BEGIN
    DECLARE x INT;
    SET x = 10;
END
```

Declarations go before executable statements in the block.

Error 6: Confusing `SET` and `RETURN`

`SET`:

```
SET total = price * quantity;
```

stores a value in a variable.

`RETURN`:

```
RETURN total;
```

sends the result back to the caller.

Error 7: Thinking `DETERMINISTIC` performs the calculation

It does not.

This:

```
DETERMINISTIC
```

describes the function.

This:

```
RETURN salary * 12;
```

performs the calculation.

---

47. How a UDF Executes

Suppose we call:

```
SELECT calculate_total(500, 3);
```

Execution conceptually looks like:

```
            Function Call
                  ↓
       calculate_total(500,3)
                  ↓
          price = 500
       quantity = 3
                  ↓
          500 × 3
                  ↓
              1500
                  ↓
              RETURN
                  ↓
            SQL Result
```

---

48. UDF Execution With a Table

Suppose:

```
SELECT employee_name, annual_salary(salary)
FROM employees;
```

MySQL evaluates the function for the relevant row values.

Conceptually: